```
# Mac
export GITHUB_SHA=$(git rev-parse HEAD)
export GITHUB_BRANCH=<MAIN_BRANCH>
# Windows
set %GITHUB_SHA%=$(git rev-parse HEAD)
set %GITHUB_BRANCH=<MAIN_BRANCH>

npm run publish:pact
```

You might notice a similar output message in figure 6.6, which states that the contract wasn't successfully uploaded to PactFlow because of versioning problems. When contract changes are made, Pact recommends a unique version number to avoid any unreliable results with contract verification.

```
 Next steps:
   * Configure separate GraphQLProvider pact verification build and webhook to trigger it when the pact
 content changes. See https://docs.pact.io/go/webhooks
Cannot change the content of the pact for MoviesAPI version 1.0.0 and provider MoviesAPI, as race condit
ions will cause unreliable results for can-i-deploy. Each pact must be published with a unique consumer
version number. Some Pact libraries generate random data when a concrete value for a type matcher is not
 specified, and this can cause the contract to mutate - ensure you have given example values for all typ
e matchers. For more information see https://docs.pact.io/go/versioning
```

Figure 6.6 Test output of trying to upload a new contract to PactFlow. PactFlow recommends having a unique consumer version every time the contract is updated.

To fix this problem, we'll update the `--consumer-app-version` for now, but in later chapters, we'll show you how to use better versioning. This will normally be the commit number from version control in real-life projects. Go ahead and update the version manually as part of the `publish:pact` command found in package.json:

```
    "publish:pact": "pact-broker publish ./pacts --consumer-app-
version=1.0.1 --tag=main --broker-base-url=$PACT_BROKER_BASE_URL
--broker-token=$PACT_BROKER_TOKEN"
```

Once the `publish:pact` command has the updated version number, save the file, and run the command again. You should now see a similar output as shown in figure 6.7.